Spring y su funcionamiento

Desarrollo Entorno Servidor

¿Qué es Spring?

Spring es un framework ligero para el desarrollo de aplicaciones en Java, bajo esto surgen dos puntos interesantes.

Primero, Spring puede ser utilizado para hacer cualquier tipo de aplicación Java (por ejemplo, aplicaciones de escritorio, web o JEE), a diferencia de otros frameworks (como Apache Struts, que está limitado a aplicaciones web.)

Segundo, el que sea ligero no está realmente referido ni al numero de clases o al tamaño de la distribución, pero prefiere definir el principio de la filosofía Spring como un todo – que es, mínimo impacto. Spring es ligero en el sentido que tu tienes que hacer poco, algún pequeño cambio en el código de tu aplicación para beneficiarte de los cambios de Spring Core, si algún momento tuvieses que parar de utilizar Spring en algún momento lo podrías realizar de manera muy simple.

Proyectos de Spring

Spring es modular por diseño, para cualquier necesidad que tenga tu arquitectura quizás existe un proyecto Spring que te ayude en ello.

Spring es modular por diseño, para cualquier necesidad que tenga tu arquitectura quizás existe un proyecto Spring que te ayude en ello. Entre ello destacamos:

- **SPRING BOOT:** proporciona una manera rápida de construir aplicaciones. De esta manera nos permite centrarnos más en la lógica de negocio y menos en la infraestructura.
- **SPRING FRAMEWORK:** es un modelo comprensivo de programación y configuración para aplicaciones basadas en JEE para cualquier plataforma.
- **SPRING CLOUD:** proporciona herramientas para desarrolladores para rápidamente construir alguno de los patrones comunes para sistemas distribuidos.

Spring Boot

Spring Boot es un framework que nace con la finalidad de simplificar aun más el desarrollo de aplicaciones basadas en el ya popular framework Spring Core. Spring Boot busca que el desarrollador solo se centre en el desarrollo de la solución, olvidándose por completo de la compleja configuración que actualmente tiene Spring Core para poder funcionar. Spring va a manejar las infraestructuras para que nosotros nos enfoquemos en la aplicación.

Ejemplos de cómo la plataforma Spring te puede ayudar en tu trabajo de desarrollador:

- Hacer un método Java que realice una transacción en la base de datos sin tener que lidiar con el API de transacciones.
- Convertir un método Java en un punto de entrada de peticiones HTTP sin tener que manejar el API de Servlets.
- Convertir un método Java en un manejador de mensajes sin tener que lidiar con el JMS API.

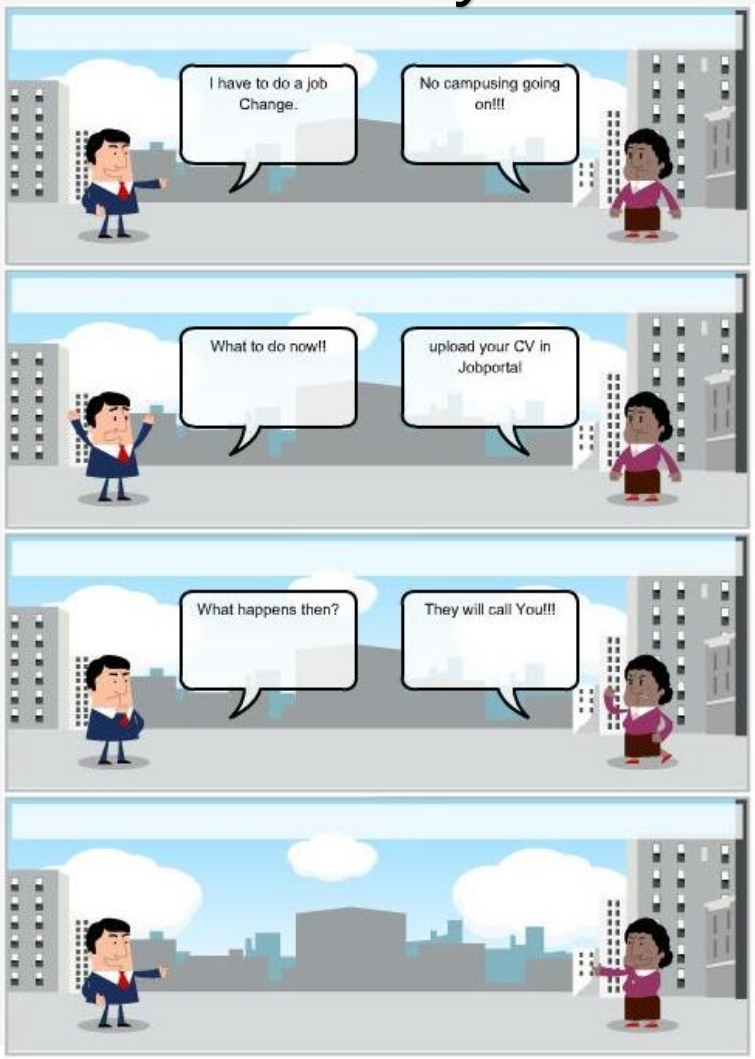
Spring y la Inyección de dependencias.

Como antes se introdujo Spring ha sido creado para liberar al programador de tareas de configuración de la infraestructura.

Para poder llevar esto acabo Spring se apoya en muchas técnicas y patrones siendo una de las principales del Framework **el patrón de Inyección de dependencias.**

El patrón de **Inyección de dependencias (DI)** que es un subtipo del patrón de **Inversión de Control (IoC)** o **Patrón Hollywood** («No nos llames, nosotros le llamaremos»).

Patrón Hollywood



Spring- IOC y DI

En la programación tradicional nosotros asignamos objetos estáticamente a otros. El programador crea el flujo del programa.

```
public class BookDaoFactoria {
    public static BookDao getDAO() {
        if (baseDatos.equals("oracle")) {
            return new BookDaoOracleImple(parametrosConexion);
        }else if (baseDatos.equals("mysql")) {
            return new BookDaoMysqlImple(parametrosConexion);
        }
    }}
```

Con Inversión de Control el flujo del programa es creado por el compilador y este lo determina en función de la interacción existente entre los objetos definidas por abstracciones

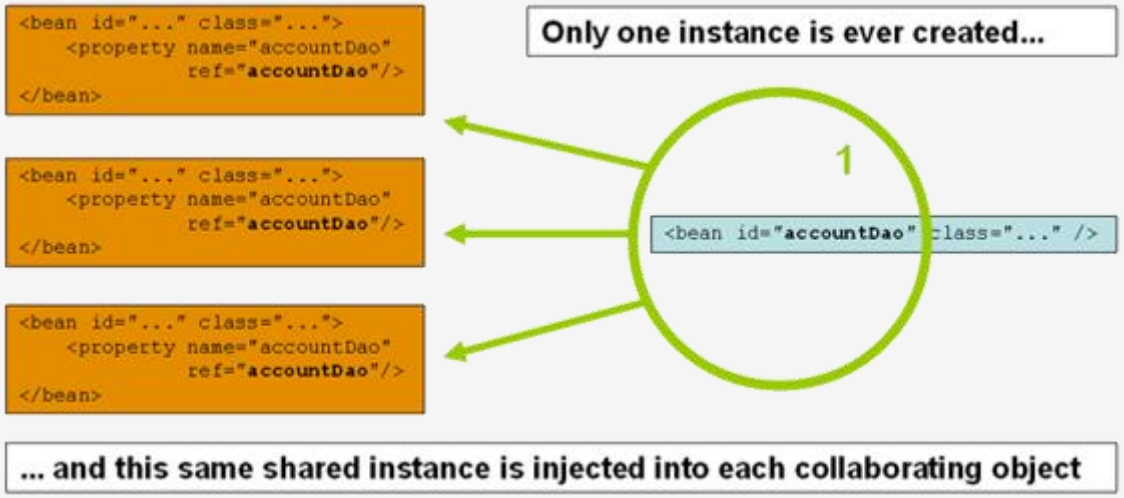
```
@Controller
public class BookController {

@Autowired
private BookDao bookdao;
```

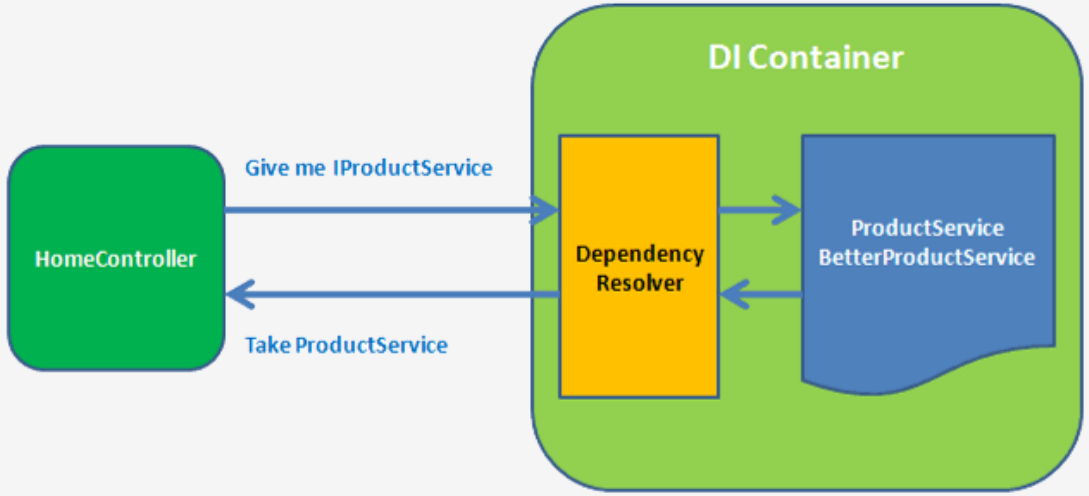
- En estas imágenes se puede ver el enfoque tradicional vs la inversión de control.
  - En el primer caso se le asigna la implementación del DAO.
  - En el segundo se llama al DAO y mediante @Autowired es Spring el que escoge la implementación más adecuada.

Spring- IOC y DI

La **Inversión de Control (IoC)** en Spring se basa en:

- Un Contenedor que maneja objetos por ti **(AplicationContext)**, este contenedor se formar con la declaración de clases en un archivo XML o de emplear anotaciones sobre las clases
- El contenedor generalmente controla la creación de estos objetos. Por decirlo de alguna manera, el contenedor hace los “new” de las clases java para que no los realices tu.
- El contenedor resuelve dependencias entre los objetos que contiene.





https://blog.auriboxtraining.com/java/introduccion-la-ioc/

¿Que son los beans?

Un **bean** es una objeto que el contenedor de Spring instancia, ensambla y maneja.

¿Deberíamos configurar todos los objetos de nuestra aplicación como beans?

Según la documentación de Spring, nosotros solo debemos definir beans para la capa de servicio, para el DAO, presentación de objetos, infraestructura ….

Tampoco debemos especificar la configuración de cada objeto en el contendor. Es responsabilidad de los DAO y de la lógica de negocio crear y cargar sus objetos del dominio.

Configuración de los Beans

Los beans pueden configurados de 3 modos:

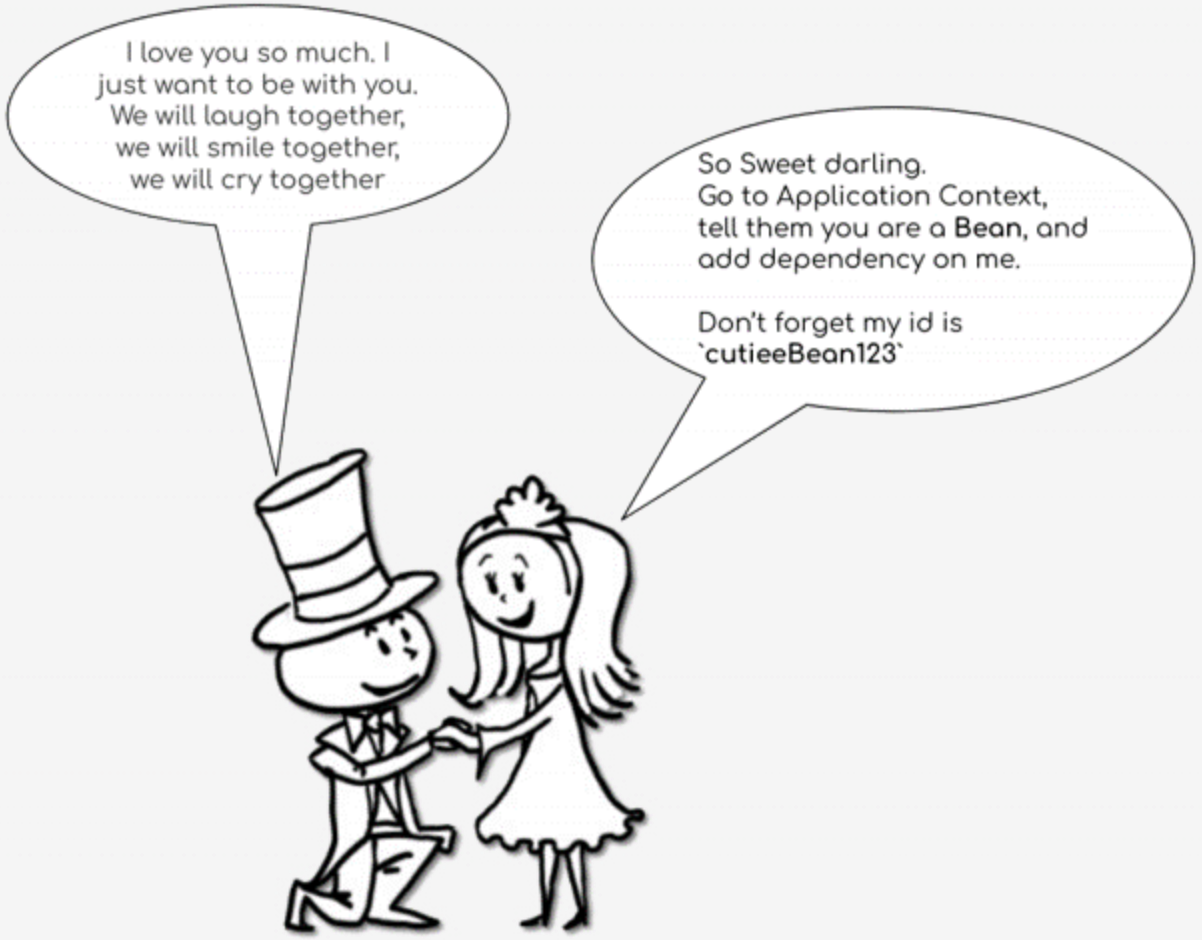
1.-Configuración con código Java utilizando la anotación **@Bean** indicas el uso de un bean. Además las clases anotadas con **@Configuration** indican que se realiza la configuración de beans.

2.-Configuración a través de anotaciones Java como **@Service, @Repository, @Model**…

3.- Configuración a través de xml, se declaran en el account-bean-config.xml

```
<bean id="accountService" class="com.baeldung.applicationcontext.AccountService">
        <constructor-arg name="accountRepository" ref="accountRepository" />
</bean>
```

Spring.- IOC y DI



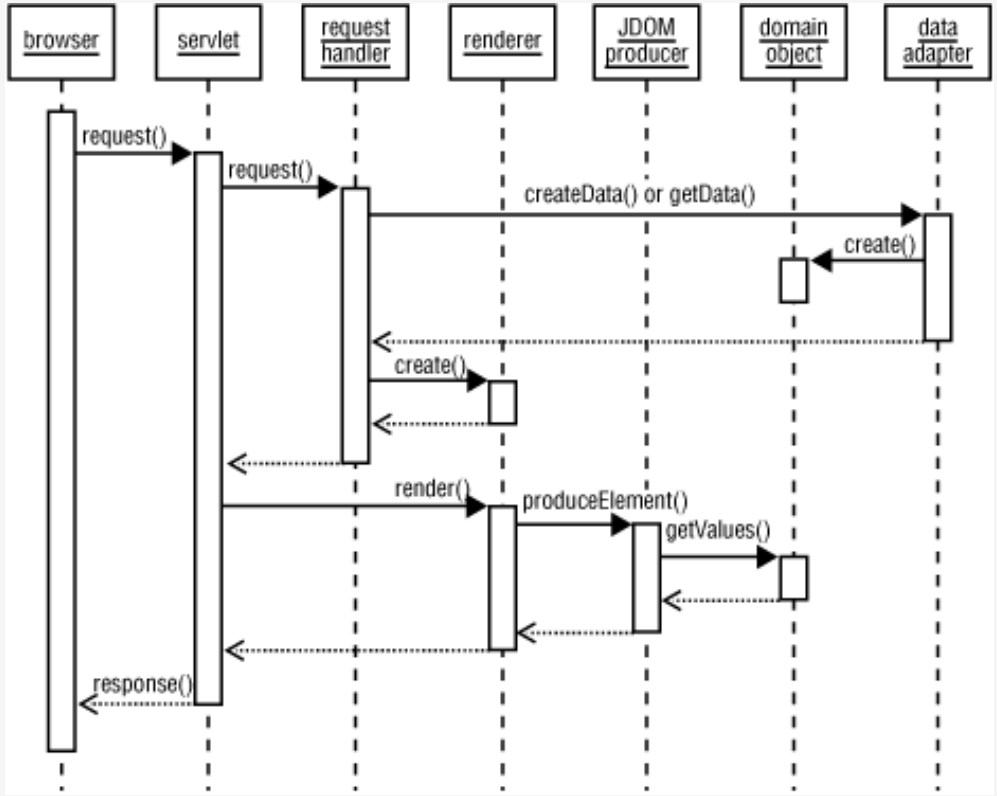
Como Spring maneja las peticiones.

Lo primero hay que tener claro que son los procesos e hilos.

- **Procesos:**
  - Las instrucciones de un programa destinadas a ser ejecutadas por el microprocesador.
  - Su estado de ejecución en un momento dado, esto es, los valores de los registros de la CPU para dicho programa.
  - Su memoria de trabajo, es decir, la memoria que ha reservado y sus contenidos.
- **Hilo, hebra o Thread:**
  - Subprograma en ejecución que comparte la imagen de memoria y otros recursos del proceso padre con otros hilos.

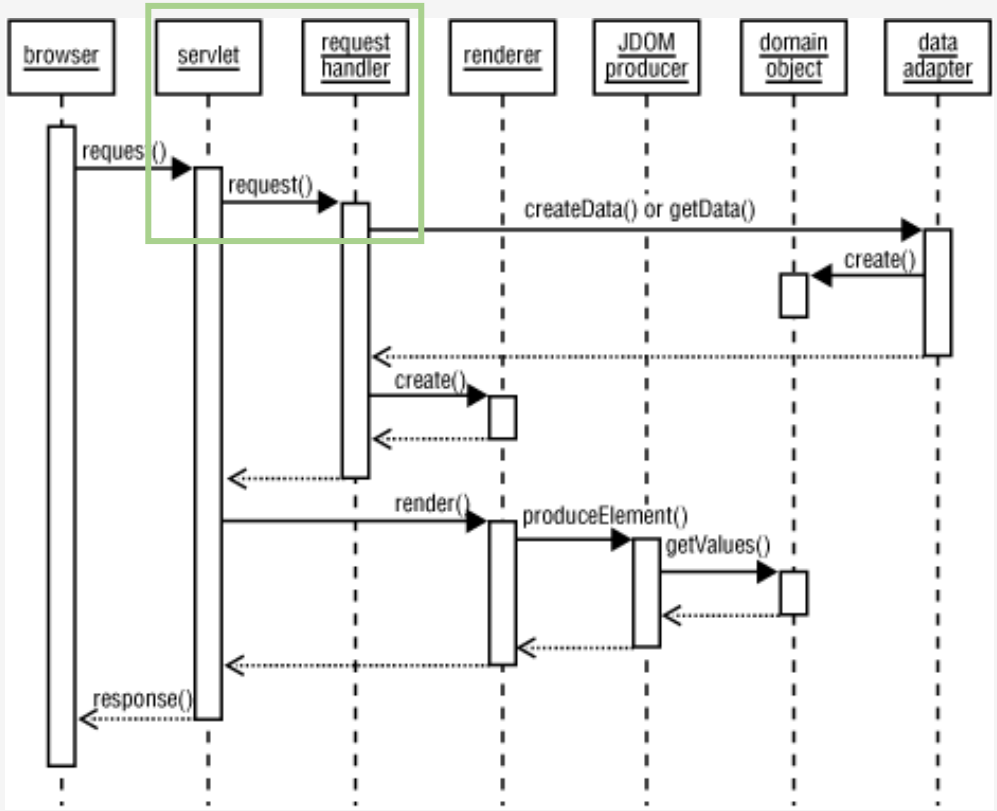
Hilo vs Thread

Simplificando el Thread es solo una conjunto de instrucciones y que es creado por un Proceso padre, con el que comparte espacio de memoria. El thread es destruido por si mismo o por el padre. De ello entendemos como funcionan los servlets.



Hilo vs Thread

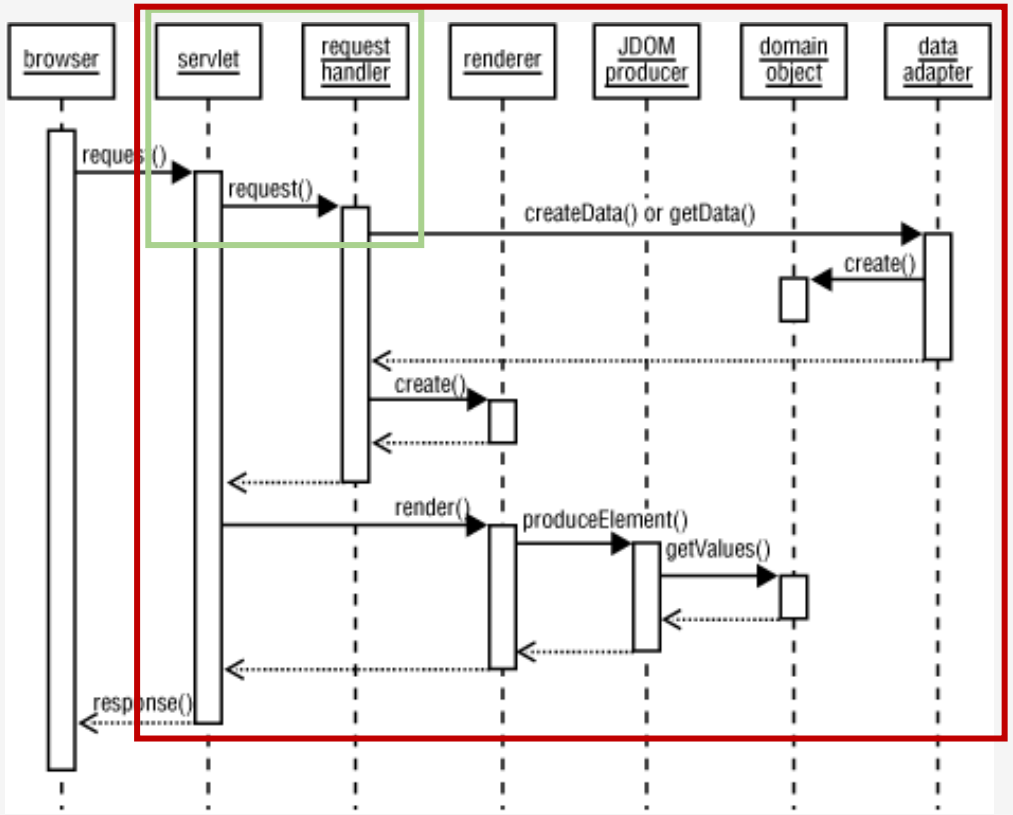
Simplificando el Thread es solo una conjunto de instrucciones y que es creado por un Proceso padre, con el que comparte espacio de memoria. El thread es destruido por si mismo o por el padre. De ello entendemos como funcionan los servlets.



De la imagen anterior podemos entender el proceso padre Spring genera una un Thread por cada petición que recibe.

Hilo vs Thread

Simplificando el Thread es solo una conjunto de instrucciones y que es creado por un Proceso padre, con el que comparte espacio de memoria. El thread es destruido por si mismo o por el padre. De ello entendemos como funcionan los servlets.



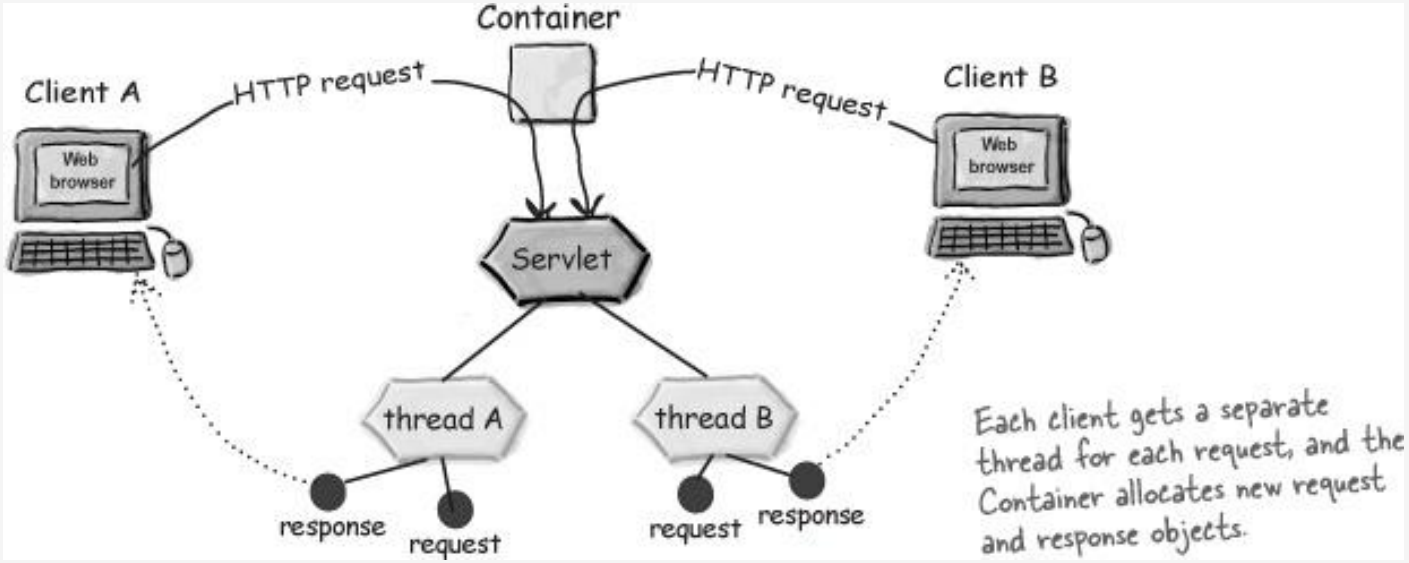
De la imagen anterior podemos entender el proceso padre Spring genera una un Thread por cada petición que recibe.

Este Thread dura lo que dura la petición, de manera que una vez que el servidor envía las respuesta el Thread se elimina junto con los datos creados del usuario.

Peticiones

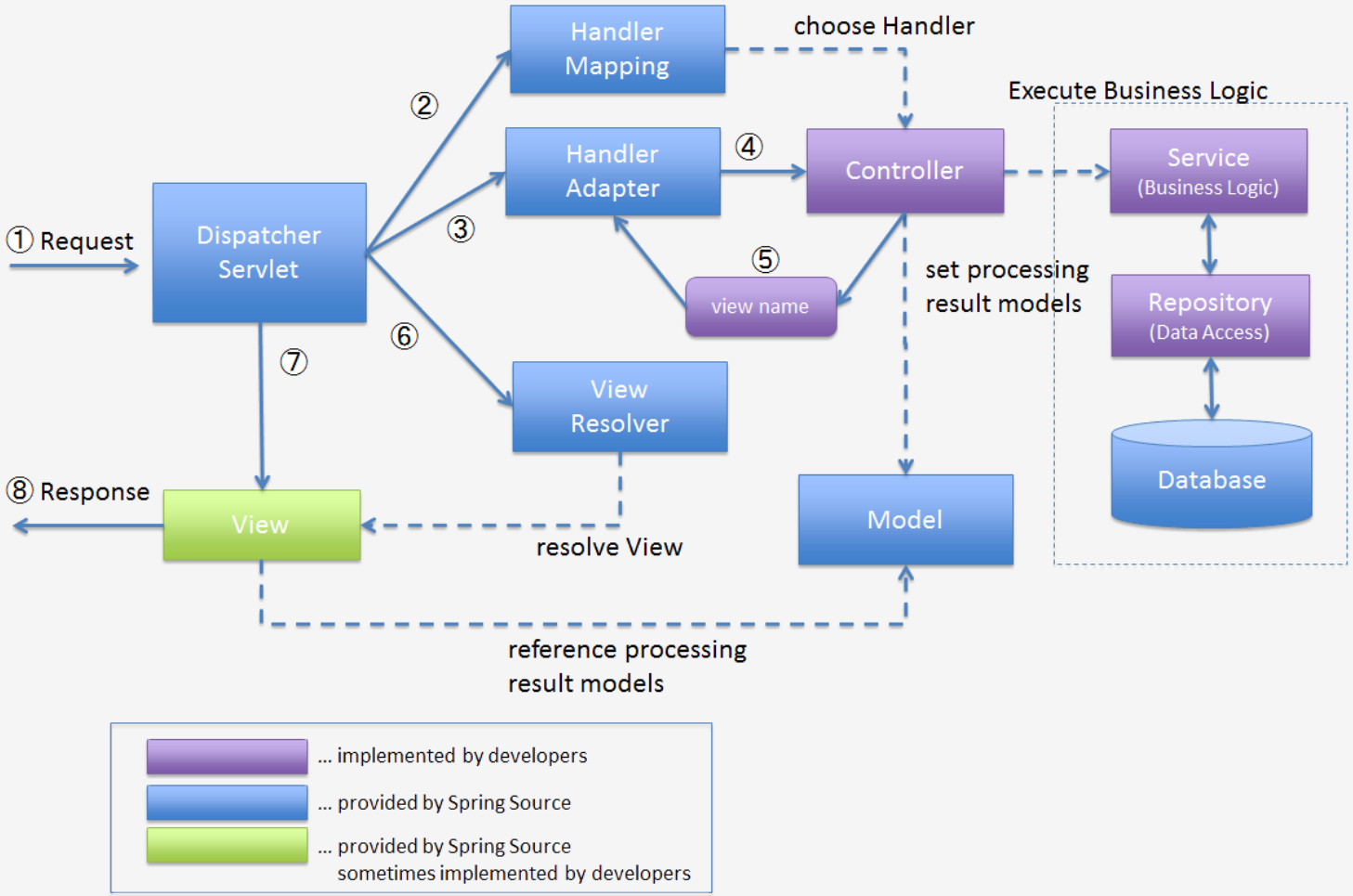
Como cada petición es procesada en el servidor por un thread. Este Thread genrea un instancia de cada clase que necesita. Una vez que el therad muere estos datos son borrados por el recolector de basura.

Por esto es imposible almacenar datos entre peticiones sin hacer uso de la Session, las Cookies y BD.



Procesamiento de peticiones

1º DispatcherServlet recibe la petición
2º DispatcherServlet asigna la tarea de escoger el **Controlador** adecuado al **HandleMapping**. **HandlerMapping** devolverá el Controlador adecuado en función de la URL al **DispatcherServlet**.
3º DispatcherServlet asigna la tarea de ejecutar la lógica de negocio del Controlador a **HandlerAdapter**.
4º HandlerAdapter llama a la lógica de negocio del Controlador.
5º El controlador ejecuta la lógica de negocio. Establece el resultado en el Modelo y devuelve el nomre de la vista al **Viewresolver**.
6º DispatcherServlet asigna la tarea de encontrar la vista al **ViewResolver**. El **ViewResolver** devuelve la Vista mapeada al nombre de la misma.
7º DispatcherServlet asigna el proceso de renderizar la vista a la vista devuelta.
8º La vista renderiza los datos del Modelo y lo devuelve en forma de Respuesta.



https://terasolunaorg.github.io/guideline/5.0.x/en/Overview/SpringMVCOverview.html